ATA 016 — Persistência em banco de dados SQLite

> **Data:** 25/09/2026
> **Projeto:** Plataforma local de gestão jurídica e automação assistida
> **Participantes:** Contratante/advogado; estagiário de Engenharia de Software (IA assistida)
> **Referência:** ATA 015 e `docs/modelo-de-dados.md`
> **Carga horária da semana (21/09 a 25/09):** 30 horas

Contexto

Até a ATA 015, as minutas existiam apenas durante a sessão: o formulário gerava Markdown/DOCX e nada era guardado. O requisito definido na ATA 002 ("o sistema deverá salvar as versões e os arquivos produzidos") exige persistência. Esta semana implementa a primeira camada de banco de dados a partir do modelo de entidades proposto.

Decisões

1. **SQLite** como banco inicial: arquivo único, sem serviço a instalar, adequado ao ambiente local e ao volume previsto (cerca de 60 processos). A troca por outro SGBD continua possível porque o acesso fica concentrado em `services/`.
2. **Migrações versionadas** (`PRAGMA user_version`): cada alteração de esquema vira uma nova versão, aplicada em transação, sem editar as anteriores.
3. **Valores monetários em centavos inteiros** (`valor_centavos`) em vez de decimal, para evitar erros de arredondamento. Ajuste em relação ao modelo conceitual, registrado em `docs/modelo-de-dados.md`.
4. A minuta é armazenada como `documento` (tipo `manifestacao_simples`), vinculada a `processo` e `cliente`; os campos do formulário ficam em JSON e o Markdown gerado é guardado junto, como fonte rastreável.
5. O banco fica em `data/` e **não é versionado no Git** (`.gitignore`).

Entrega realizada

- `services/database.py`: esquema v1 com as sete entidades (usuário, cliente, processo, prazo, tarefa, documento e financeiro), chaves estrangeiras ativas, restrições `CHECK` e índices.
- `services/minutas.py`: validação dos cinco campos (obrigatórios, com limite de tamanho), geração do Markdown e gravação da minuta (cria cliente e processo quando ainda não existem).
- `tests/test_database.py` e os primeiros testes de `tests/test_minutas.py`.

Validação técnica

Foram escritos 9 testes automatizados, todos aprovados: migração idempotente, existência das entidades, rejeição de processo sem cliente, perfil inválido e número de processo duplicado, criação de minuta com reaproveitamento de cliente/processo, campos inválidos e uma tentativa de injeção de SQL tratada como texto comum.

```
python -m unittest discover -s tests -t .
```

Anexo técnico — trecho comentado

```
def migrar(conexao):
    atual = conexao.execute("PRAGMA user_version").fetchone()[0]   # versão gravada no
arquivo
    for versao, script in enumerate(MIGRACOES, start=1):
        if versao <= atual:
            continue                                                # já aplicada
        conexao.executescript(f"BEGIN;\n{script}\nPRAGMA user_version = {versao};\nCOMMIT;")
```

Cada versão roda dentro de uma transação: se o script falhar no meio, nada é aplicado e a versão não muda.

Limitações e riscos

- Ainda não há login: qualquer chamada local grava no banco. Por isso só dados fictícios são usados.
- O número do processo é texto livre nesta etapa; a validação do padrão CNJ ficou para o módulo de cadastros.
- Risco: perda do arquivo do banco. Mitigação planejada: rotina de backup.

Pendências para o contratante

- Validar se o modelo de entidades cobre o fluxo atual do escritório (em especial prazos e financeiro).

Próximo passo

Implementar autenticação, perfis de acesso e trilha de auditoria.

Documento de acompanhamento do projeto. Não substitui documentos oficiais da UTFPR nem instrumentos jurídicos do escritório.